

Input Validation & Output Encoding – or how to defeat injections

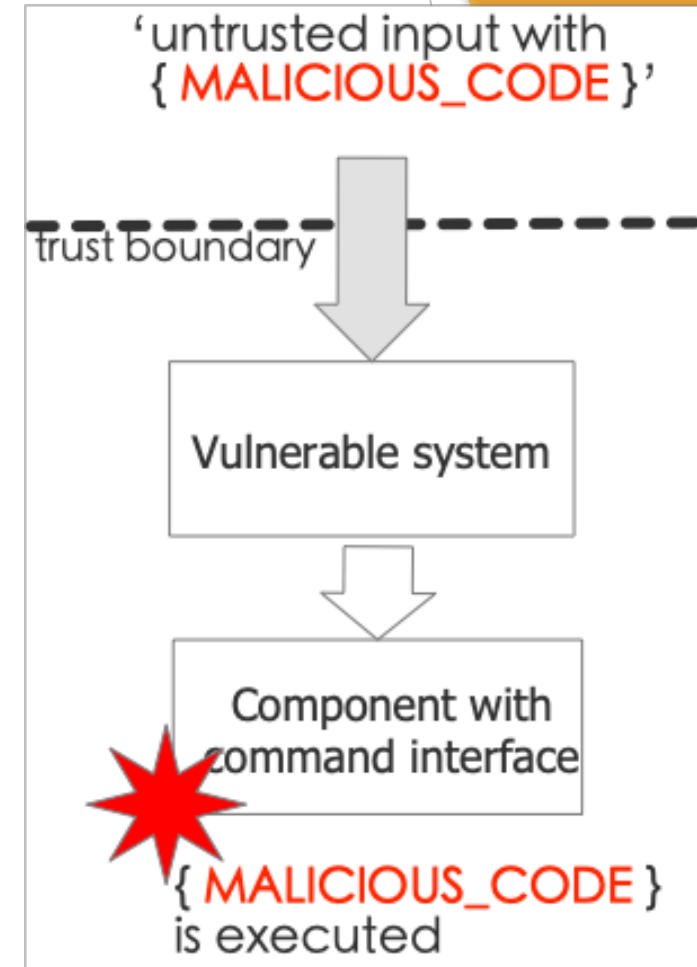
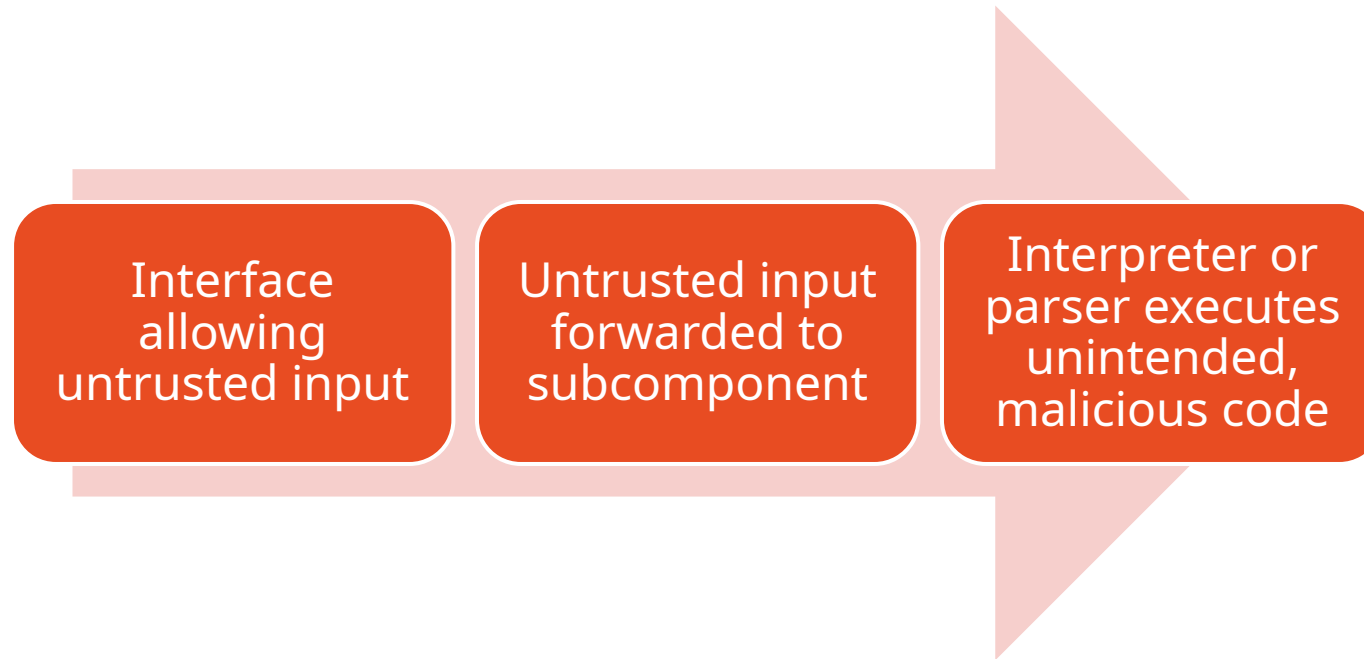
Secure Programming

Michael Lehmann



IT SECURITY

Injection: Conceptual Model



https://owasp.org/www-community/Injection_Theory

Many different interfaces can be vulnerable

What do you think these are?

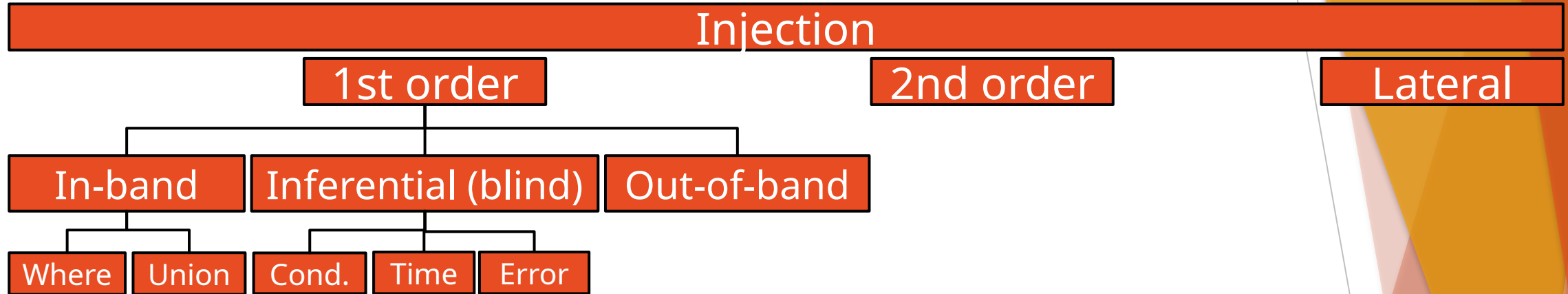
SQL Injection



Quite a few fell victim

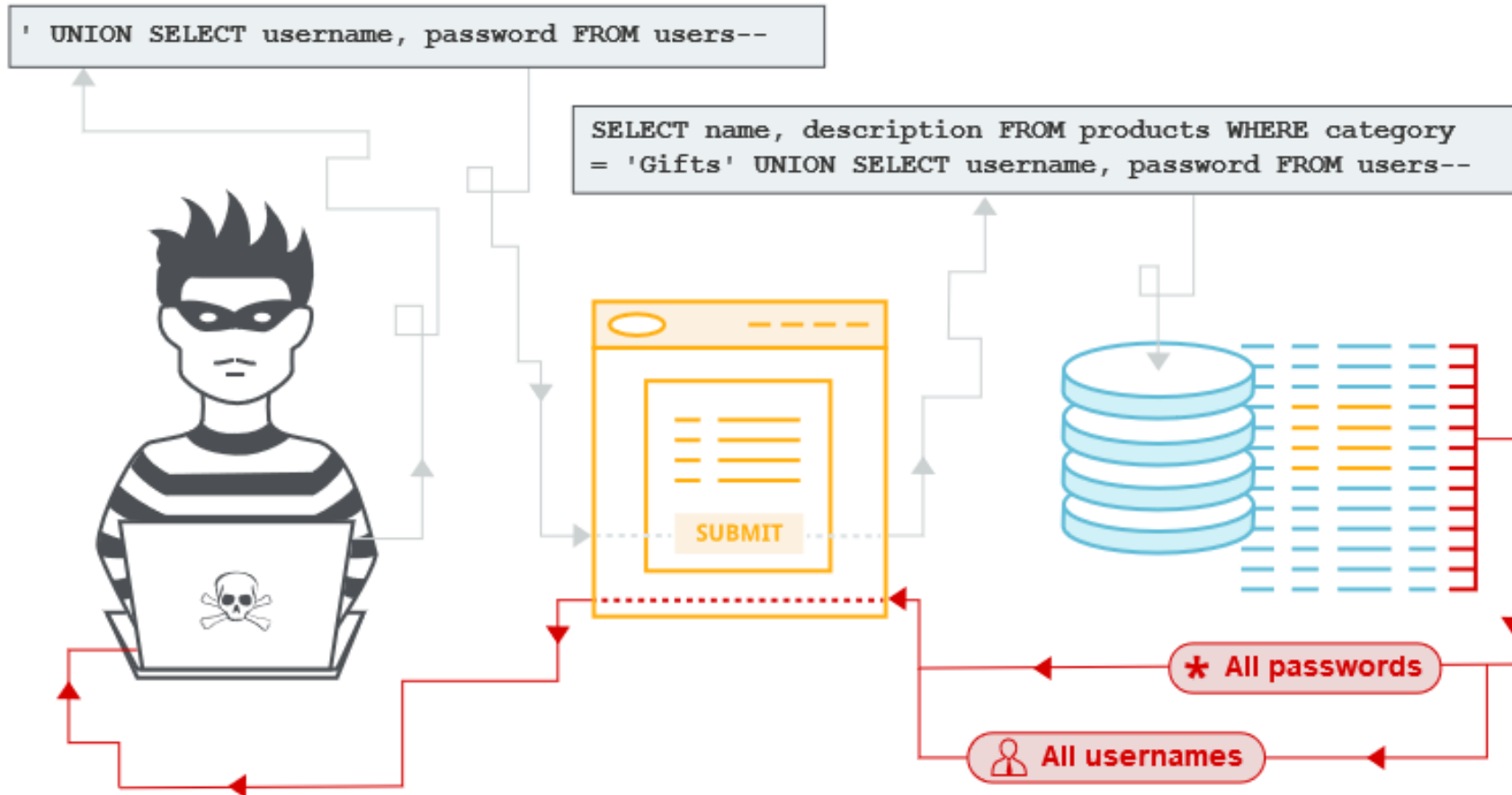
- ▶ Some high profile victims
 - ▶ Adobe (2012): 150k accounts and Passwords
 - ▶ Yahoo (2012): 450k passwords
 - ▶ LinkedIn (2012): 6.5 Mio accounts and passwords
 - ▶ TalkTalk (2015): 4 Mio personal details and credit cards
 - ▶ Wordpress (2017): user info, passwords
 - ▶ GoDaddy (2017): admin account
 - ▶ Equifax (2017): 145 Mio personal data
- ▶ SQL Injection hall of shame
<https://codecurmudgeon.com/wp/sql-injection-hall-of-shame/>

Types of injection attacks

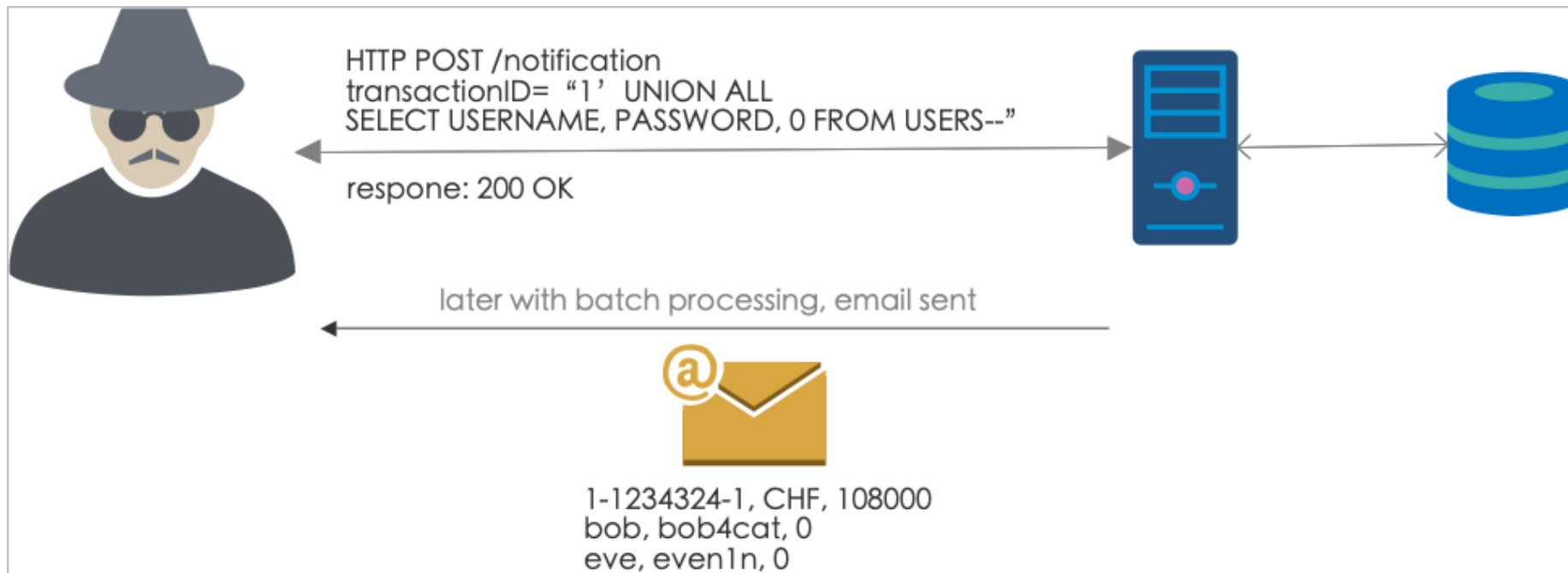


- ▶ In-band: results directly returned through same channel
- ▶ Inferential/blind: different responses reveal information
- ▶ Out-of-band: results returned through different channel, e.g. e-mail
- ▶ 2nd order: payload is not sent to the target directly
- ▶ Lateral: shifting or transformation of the representation of one datatype to another
 - ▶ See [Oracle](#)

In-band, if results come back the same way



Out-of-Band if through different channel



Blind if no result is returned



Different methods can be used

- ▶ «Basic» **conditional** if the attacker has an account:

```
/balances?accountId=123' union select * from users where username='Admin'  
and substring(Password, 1, 1) = 'e' --
```

- ▶ Or **time based** if the attacker does not have an account:

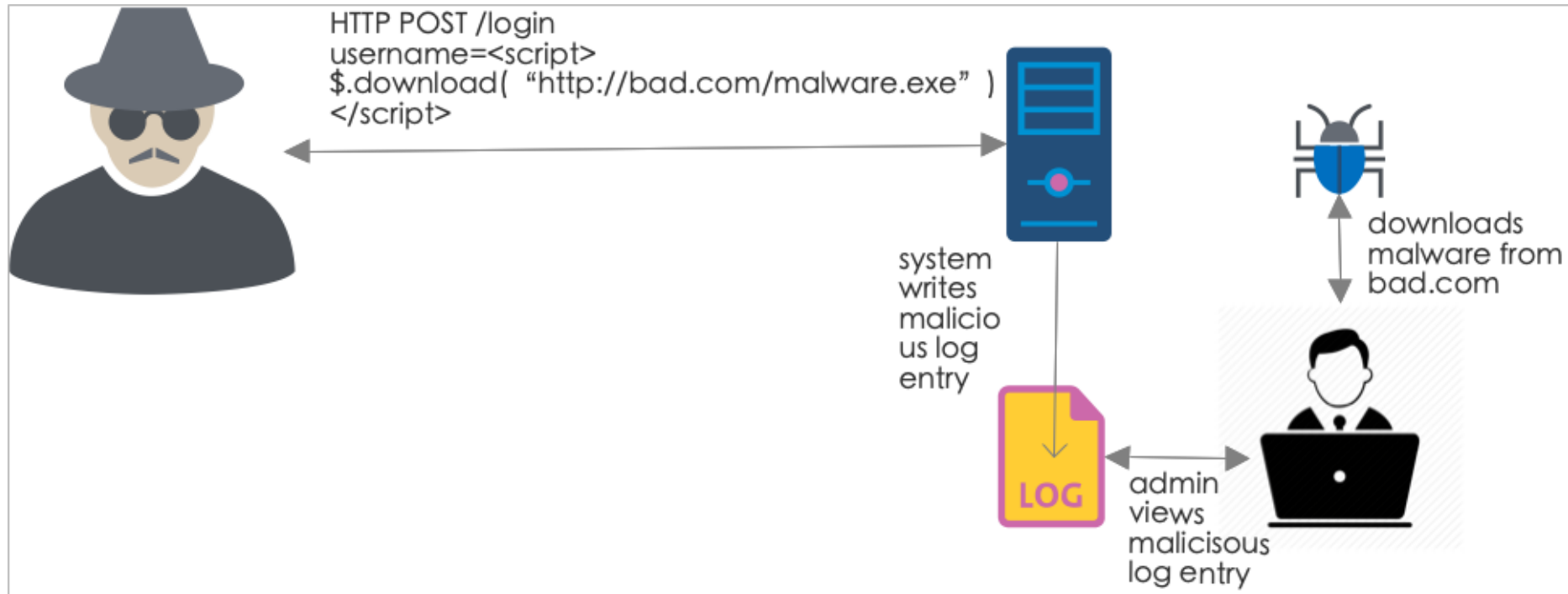
```
/balances?accountId=123' union select * from users where username='Admin'  
and substring(Password, 1, 1) = 'e' and sleep(10) --
```

- ▶ Or via **causing errors**:

```
/balances?accountId=123' union select case when (username = 'Admin'  
and substring(password, 1, 1) = 'e') then 1/0 else null end from users --
```

- ▶ Details depend on exact situation, e.g. exact database system

2nd order if target is another system



SQL Injection techniques

WHERE clause tampering

- Modify the where clause
 - e.g. by adding 'OR 1=1--
- Access more data of the same table and columns

UNION query

- Append results
 - Accessing columns of another table
- ```
SELECT NAME, DESCRIPTION, PRICE from PRODUCT
where NAME LIKE '%x' UNION ALL SELECT USERNAME,
PASSWORD, 0 FROM USER --%'
```

# How to prevent Injections

- Parametrized API (prepared statements)
- Input validation / sanitisation
- Encoding
- Least privilege
- Configuration

# Reject input if it does not meet the criteria

- Originates from a legitimate system or user
- Has reasonable size
- Lexically valid
- Syntactically valid
- Semantically valid

# Whitelisting is the most secure approach



- Data type validators



- RegEx (use with care: [OWASP ReDoS](#))



- Whitelisting
- Blacklisting



- Implementations with schemas or source code

# Example 1: Authentication bypass

**Authentication logic** using SQL query with *input1* and *input2* from the user (input login page) to validate the user's password

```
SELECT USERNAME FROM USER
WHERE USERNAME='{input1}' AND PASSWORD='{input2}'
```

If the value of the user supplied parameter *input1* is "Bob' --" , then the query returns "Bob" and omits the password validation:

```
SELECT USERNAME FROM USER
WHERE USERNAME='Bob'-- AND PASSWORD = 'x'
```

Injection into the PASSWORD variable, i.e. the value of "x' OR USERNAME='admin':

```
WHERE USERNAME='Bob' AND PASSWORD = 'x' OR USERNAME='admin'
```



# Example 2: Data Exfiltration with UNION query

Search function in a product catalog using the with LIKE criterion.

```
SELECT PRODUCT_NAME, PRICE, DESCRIPTION
FROM PRODUCT WHERE PRODUCT_NAME LIKE '%{pname}%'
```

Set *pname*="X' UNION SELECT USERNAME, 0, PASSWORD FROM USER --"

The executed query will be:

```
SELECT PRODUCT_NAME, PRICE, DESCRIPTION
FROM PRODUCT WHERE PRODUCT_NAME LIKE 'X' UNION SELECT USERNAME, 0, PASSWORD
FROM USER -- %'
```

Returning all the users with passwords after products whose name ends with X

# Example 3: Data Exfiltration with time-based Blind SQL

Slightly adjusted logic in some authentication service using a query with *input1*

```
SELECT USERNAME, PASSWORD FROM USER WHERE USERNAME='{input1}'
```

In this case the parameter *input1* might be injectable, but the result is not returned by the client.

We can inject a query that sleeps for 5 seconds if a condition is met:

```
SELECT USERNAME, PASSWORD FROM USER WHERE USERNAME = 'test123'
UNION
SELECT IF (SUBSTRING(PASSWORD, 1, 1) = 'A', SLEEP(5), NULL), NULL FROM USER WHERE USERNAME = 'admin';
```

This stops the application thread for 5 second if the the first character of the 'admin' user's password is 'A'.

Having this automated, the whole database may be read, character by character.

# Example 4: XML/XPATH injection

XPATH: Extract nodes of an XML file from a path

```
<?xml version="1.0" encoding="utf-8"?>
<employees>
 <employee id="AS789" firstname="John" lastname="Doo" annualsalary="70000"/>
 <employee id="AS719" firstname="Isabela" lastname="Dobora" annualsalary="90000"/>
 <employee id="AS219" firstname="Eric" lastname="Lambert" annualsalary="65000"/>
</employees>
```

- ▶ To select an employee with a given ID the below XPATH query is used  
/employees/employee[@id='EMPLOYEE\_ID']
- ▶ However, if EMPLOYEE\_ID is ' or '1'='1' then all employees are returned because the query will be  
/employees/employee[@id=" or '1'='1']
- ▶ Protection:
  - ▶ Use XML encoding before applying XPath. -> ' becomes &apos;

# Example 5: OS command injection

- ▶ Happens when calling executables on the host system

- ▶ `call("good.exe" + parameter)`
- ▶ `parameter = "goodParam; bad.exe;"`  
→ `call("good.exe goodParam; bad.exe")`

```
<?php
print("Please specify the name of the file to delete");
print("<p>");
$file=$_GET['filename'];
system("rm $file");
?>
```

- ▶ How to prevent the attack?

# Cross-Site Scripting (XSS)

# Cross Site Scripting (XSS)

- ▶ Exploits vulnerabilities in a web application to inject malicious code (**JavaScript**)
- ▶ The malicious script runs on the user's client, typically a browser
- ▶ **Confused deputy attack**
  - ▶ A command or malicious script is running under the privileges of the victim user

# Types of XSS

- ▶ **Stored XSS** (aka type I)
  - ▶ Malicious script is **stored**, e.g. in
    - ▶ Message forums
    - ▶ Visitor logs
    - ▶ Comment fields etc.
- ▶ **Reflected XSS** (aka type II)
  - ▶ User input is **returned** to the browser, e.g. in
    - ▶ Error message
    - ▶ Search result
    - ▶ Page content etc.
- ▶ **DOM based XSS** (aka type-0)
  - ▶ Manipulates DOM
  - ▶ Source and sink in browser

- ▶ In reality, the three types overlap

Where untrusted data is used		
Data Persistence	XSS	
	Server	Client
	Stored	Stored
	Server XSS	Client XSS
	Reflected	Reflected
	Server XSS	Client XSS

- ❑ DOM Based XSS is a subset of Client XSS (where the data source is from the DOM only)
- ❑ Stored vs. Reflected only affects the likelihood of successful attack, not the nature of vulnerability or the most effective defense

# Stored server-side XSS

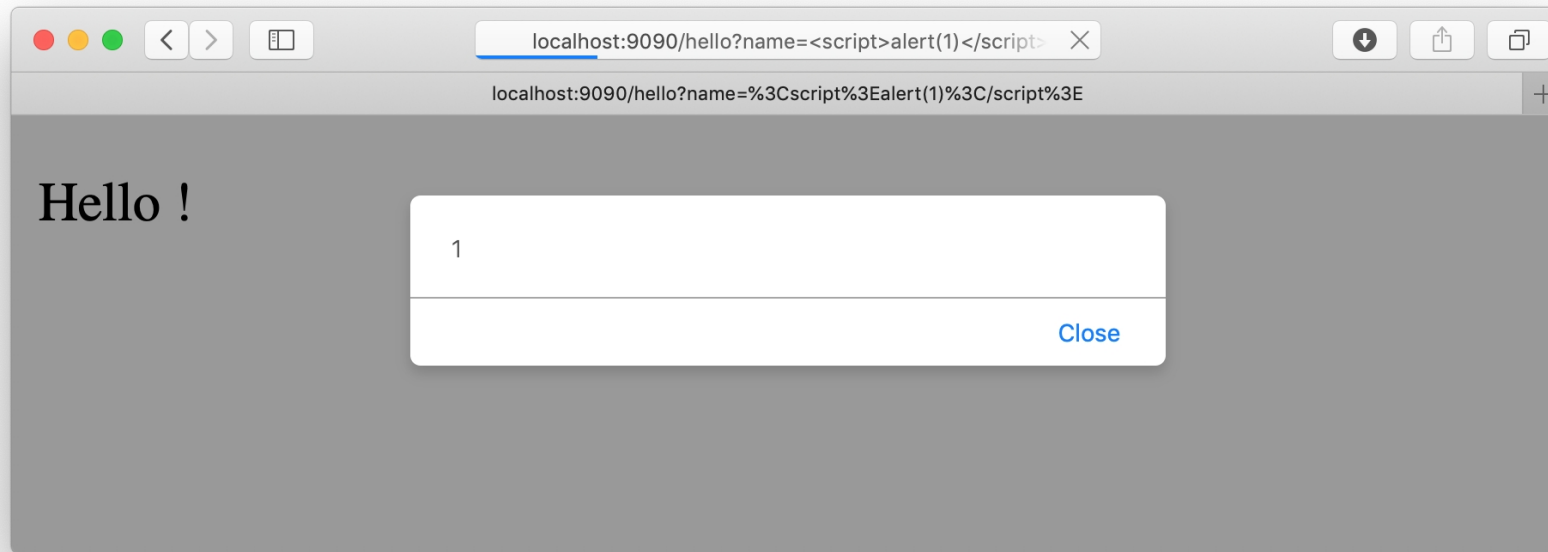
```
<!DOCTYPE html>
<html lang="en">
 <body>
 <p>Hello ${username}!</p>
 </body>
</html>
```

```
@GetMapping("/{hello}")
public String hello(Model model, Integer userId) {
 String username = PersistenceService.getUserNameFromId(userId);
 model.addAttribute("username", username);
 return "hello";
}
```



# Reflected server-side XSS

```
<!DOCTYPE html>
<html lang="en">
 <body>
 <p>Hello ${param.name}!</p>
 </body>
</html>
```



# DOM-based XSS Example

Consider this code to choose the preferred language while providing a default in the query string:

```
...
Select your language:

<select><script>

document.write("<OPTION value=1>" + document.location.href.substring(document.location.href.indexOf("default=") + 8) + "</OPTION>");

document.write("<OPTION value=2>English</OPTION>");

</script></select>
...
```

The following request will trigger a DOM based XSS:

```
http://www.some.site/page.html?default=<script>alert(document.cookie)</script>
```

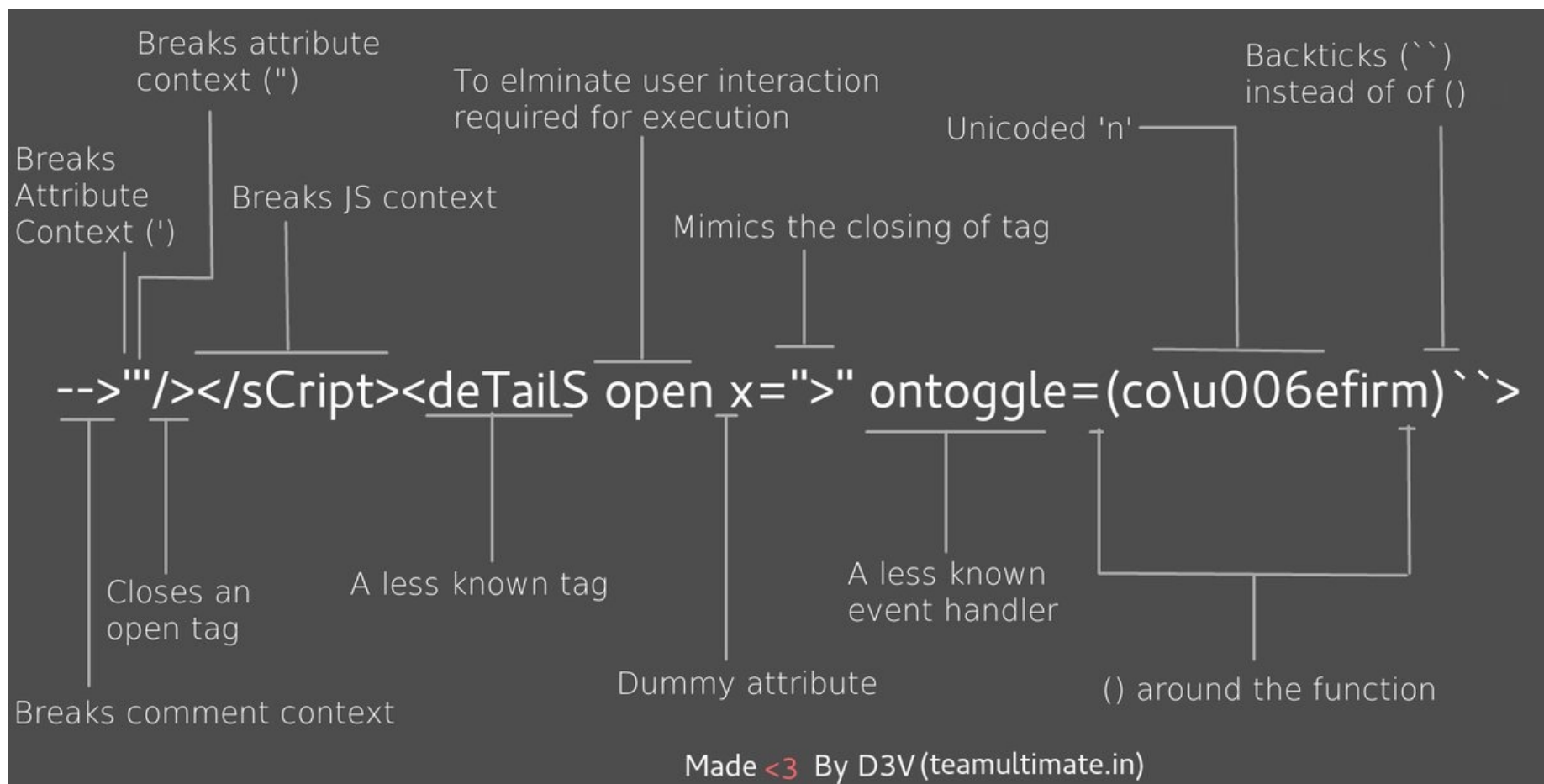
This will be sent to the server, but **not** returned, i.e. no change in the response.  
An attacker can further tune this not to send the payload to the server:

```
http://www.some.site/page.html#default=<script>alert(document.cookie)</script>
```

# XSS Protections

How can you protect against XSS? What measures would you take?

# Use Framework for Protection



# XSS Protections

## Encode output example (JSF):

- ▶ UserBean.java

```
public class UserBean{
 public String p0wned = "<script>alert('XSS')<script/>";
```

- ▶ JSF – default (good)

```
<h:outputText value="#{userBean.p0wned}" />
```

- ▶ JSF unescaped (bad)

```
<h:outputText value="#{userBean.p0wned}" escape="false" />
```

# XSS Protections

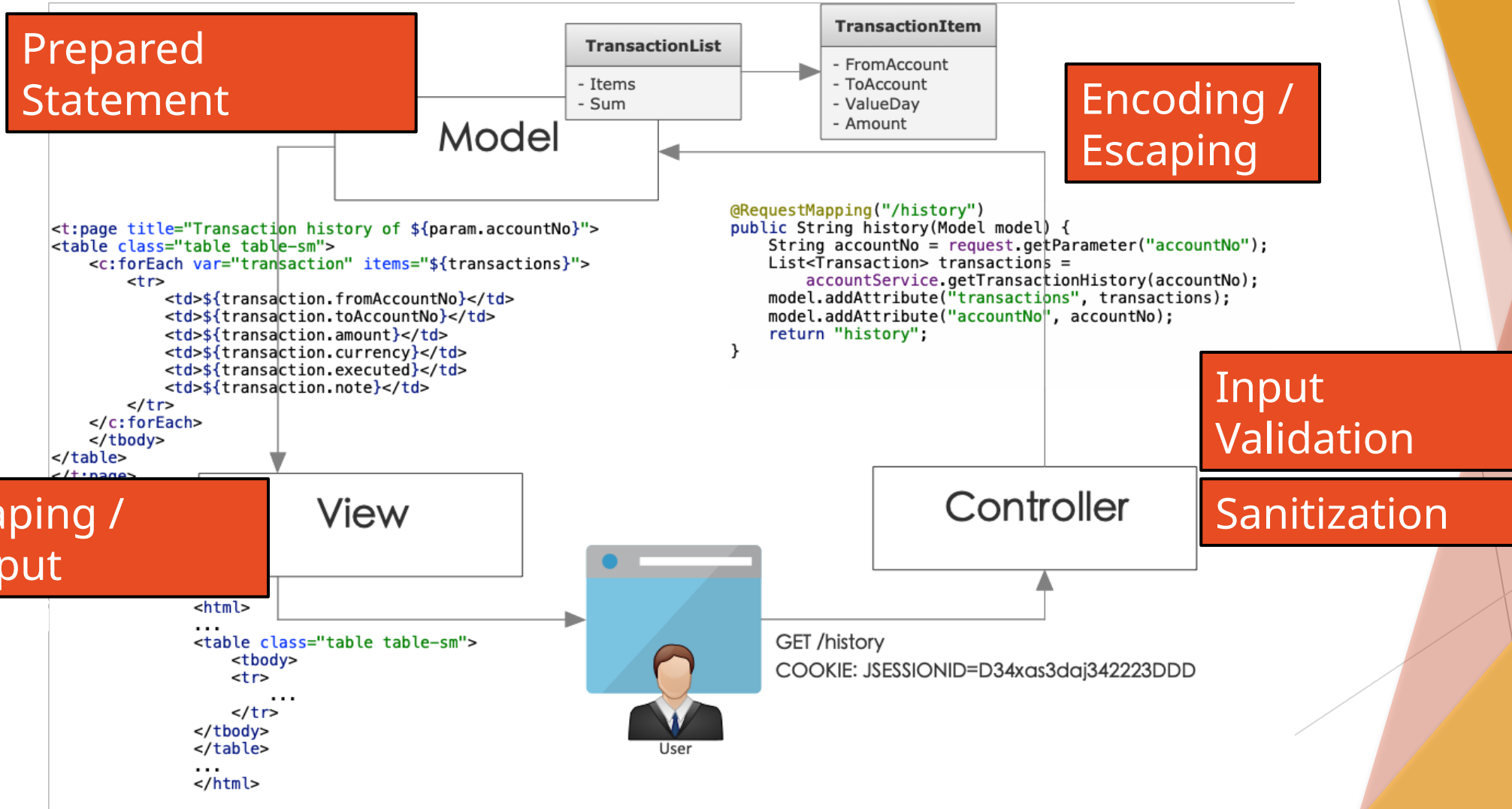
- ▶ Use **contextual** output encoding
- ▶ Server-side encode for output context and client-side encode for subcontext
  - ▶ Example using [ESAPI4JS](#)

```
//server-side encoding
var input = "<%=Encoder.encodeForJS(untrustedData)%>";
```

```
//HTML encoding is happening in JavaScript
document.writeln(ESAPI4JS.encodeForHTML(input));
```

- ▶ Read more details here:
  - ▶ [OWASP XSS Prevention Cheat Sheet](#)
  - ▶ [OWASP DOM Based XSS Prevention Cheat Sheet](#)

# Model View Control (MVC)



# Content Security Policy (CSP)

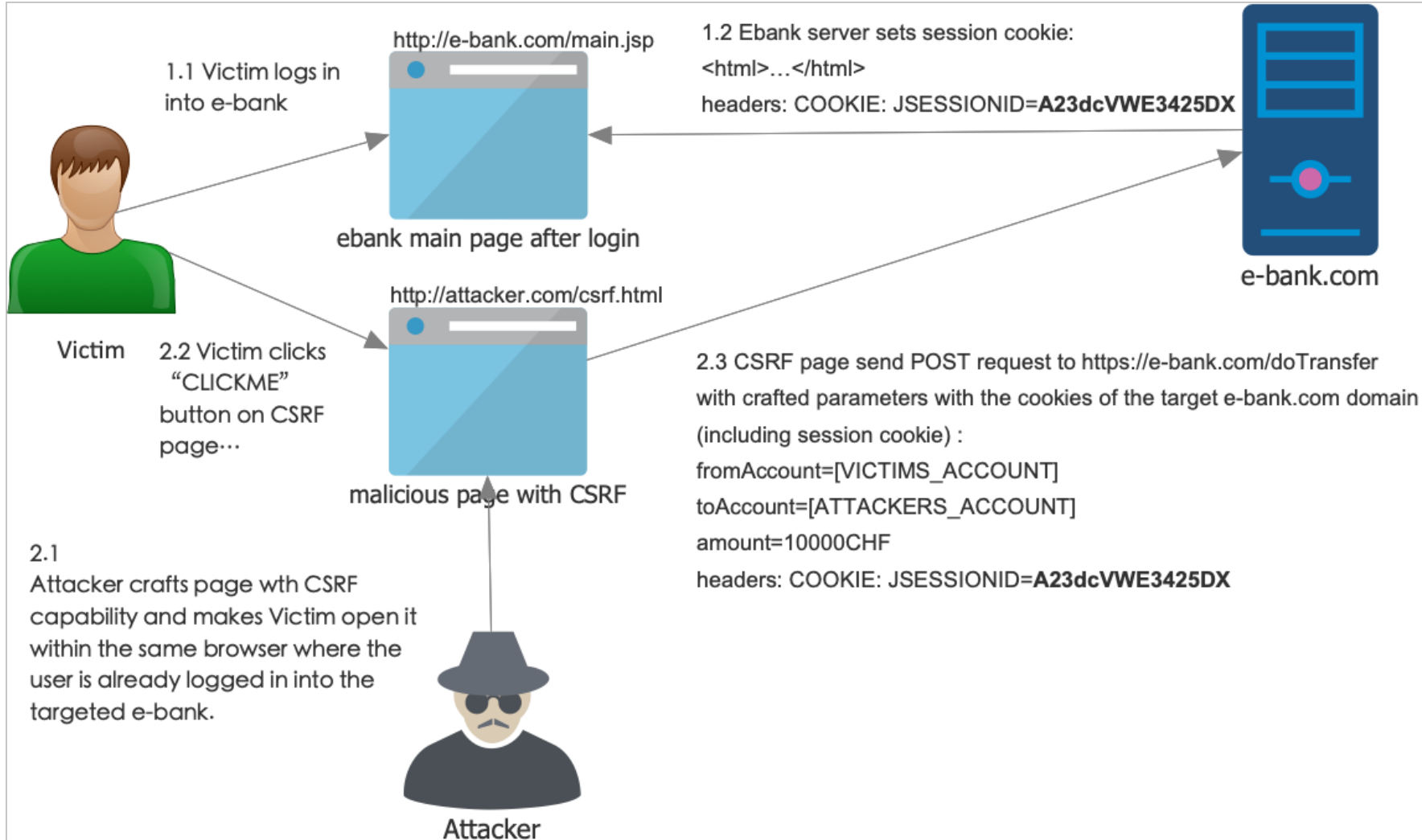
- ▶ Mechanism to control which content can be loaded by the browser
- ▶ The server sends a special header 'Content-Security-Policy' that contains directives to be applied for the application
  - ▶ Which types are allowed (css, img, html)
  - ▶ Which files are trustworthy and can be loaded, and from where (domain whitelist)
  - ▶ Violation reporting
- ▶ The browser needs to apply the requested policy (CSP compatible)

<https://content-security-policy.com>



# Cross Site Request Forgery (CSRF)

# Cross Site Request Forgery attack



# Mainly three protection options

## CSRF Token

- Randomly generated token
- Set in HTTP response body or header
- Sent with each modifying request

## Samesite Cookie

- Attribute on session cookie
- Strict or lax
- <https://web.dev/samesite-cookies-explained/>

## Double Submit Cookie

- Additional “CSRF-Token” cookie
- Client copies token as request parameter
- Server verifies both are identical

# Exercise

# Identify the problem

1. What's wrong here?
2. How can you fix the issue?

python

 Copy code

```
def fetch_records():
 user_input = input("Enter the record ID: ")
 query = "SELECT * FROM records WHERE id=" + user_input
 result = database.execute(query)
 return result
```

# Identify the problem

1. What's wrong here?
2. How can you fix the issue?

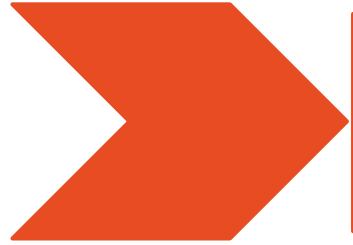
javascript

 Copy code

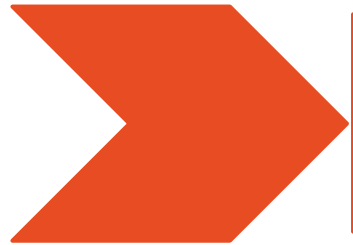
```
function displayComments() {
 let userComments = getCommentsFromDatabase(); // This function retrieve
 for (let i = 0; i < userComments.length; i++) {
 document.write(userComments[i].text);
 }
}
```

# XS Leaks

# The Web is Vulnerable – XS-Leaks



- Web content often combined from various sources
  - e.g. embedded Youtube video on Facebook



- Rendering of other content gives timing side channel
  - Information leaked



- Many other side channels
  - Input, onload, etc.

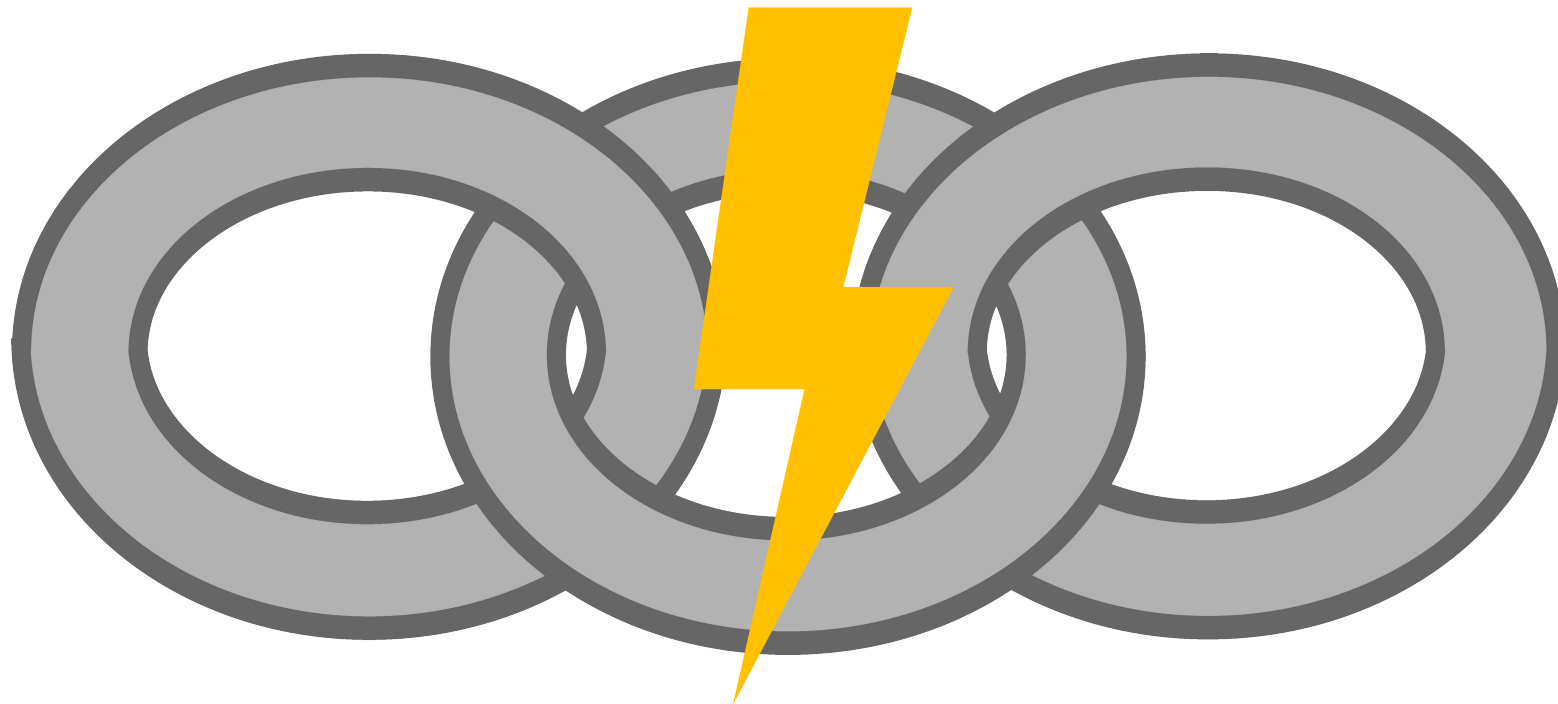


# Excercise: Attack Chain

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE html>
3 <html>
4 <body>
5
6
9 </body>
10 </html>
```

→ What could something like this be used for?

# Break Attack Chains!



XS-Leaks + CSRF + XSS

# Exercises

- ▶ <https://overthewire.org/wargames/natas/>

